



Instituto Tecnológico de San Pedro Coahuila

Ingeniería en Sistemas Computacionales

Asignatura: Fundamentos de Ingeniería de Software

U1-E02 Análisis individual de un caso

Nombre alumno: Obregon Estrada Emmanuel – Analizar el CO2

Docente: Ruth Aivi Chávez Rodriguez

CO2 - Plataforma de inscripción escolar

Resumen del problema

La plataforma escolar presenta lentitud, solicitudes duplicadas y errores en los cupos durante la mayor demanda. Además, no hay control centralizado de cambios, pruebas de carga, plan de regreso a una versión estable ni seguimiento adecuado de reportes. Las áreas también manejan cifras diferentes en los cupos, dificultando identificar el origen de los problemas.

1. Descripción y consecuencias

La plataforma escolar deja consultar materias, escoger grupos y generar comprobantes de inscripción. Aunque funcionó bien con pocos usuarios, presentó problemas al aumentar la demanda: se volvió lenta, algunas solicitudes se procesaron dos veces y algunos estudiantes recibieron comprobantes cuando el cupo ya estaba agotado. También existen fallas de proceso, ya que los cambios de requisitos llegan por diferentes medios, no hay una lista única de decisiones, no se realizan pruebas de carga, no existe un plan para regresar a una versión estable y los reportes de soporte no se clasifican ni relacionan con versiones. Estas fallas afectan a los estudiantes, quienes pueden perder tiempo o recibir información incorrecta; al equipo de desarrollo, que tiene dificultades para encontrar el origen de los errores; y a soporte, que recibe reportes sin una prioridad clara. La institución también puede enfrentar conflictos por inscripciones incorrectas y perder confianza en la plataforma. Por ello, el problema no es solo técnico, sino también de organización y proceso. ISO/IEC 25010 considera el rendimiento y la fiabilidad como aspectos importantes de la calidad del software que deben evaluarse durante su ciclo de vida.

2. Cinco problemas identificados

Problema 1: Falta de pruebas de carga.

Causa: las pruebas se realizaron con pocos usuarios y no se simuló la demanda máxima.

Consecuencia: la plataforma se vuelve lenta durante el periodo de mayor inscripción. Esto afecta la experiencia de los estudiantes y puede aumentar la cantidad de solicitudes fallidas o repetidas.

Práctica: realizar pruebas de rendimiento y carga antes del periodo de mayor demanda. Estas pruebas permiten simular usuarios y detectar problemas de capacidad y rendimiento.

Evidencia: el caso indica que no se realizan pruebas de carga y que la plataforma se vuelve lenta durante la mayor demanda.

Problema 2: Procesamiento duplicado de solicitudes.

Causa: el sistema no está preparado adecuadamente para manejar solicitudes repetidas o concurrentes.

Consecuencia: una acción del estudiante puede procesarse dos veces, provocando registros incorrectos o afectando la disponibilidad de los grupos.

Práctica: aplicar pruebas de concurrencia y control transaccional, verificando que una misma operación no genere resultados duplicados.

Evidencia: el caso señala que algunas solicitudes se procesan dos veces.

Problema 3: Falta de control consistente sobre los cupos.

Causa: el sistema permite generar comprobantes, aunque el cupo ya se haya agotado y, además, las áreas manejan cifras diferentes.

Consecuencia: pueden existir inscripciones que superen la capacidad real de un grupo, generando conflictos entre estudiantes, personal administrativo y responsables de la plataforma.

Práctica: utilizar gestión de datos y validaciones de integridad, asegurando que la información del cupo sea consistente antes de confirmar una inscripción.

Evidencia: algunos estudiantes reciben comprobantes, aunque el cupo ya se agotó.

Problema 4: Cambios de requisitos sin control centralizado.

Causa: los cambios llegan por correos y mensajes separados, sin una lista única de decisiones.

Consecuencia: diferentes integrantes pueden trabajar con requisitos distintos, aumentando las posibilidades de que se encuentren errores y tengan que volver a trabajarlo.

Práctica: aplicar gestión de requisitos y control de cambios, registrando cada modificación, decisión y responsable. ISO/IEC 25030 señala la importancia de definir, mantener y gobernar los requisitos de calidad durante el desarrollo.

Evidencia: el caso especifica que los cambios llegan por diferentes medios y no existe una lista única.

Problema 5: Falta de trazabilidad en los reportes de soporte.

Causa: soporte no clasifica los reportes por prioridad ni puede relacionarlos con una versión específica.

Consecuencia: los problemas críticos pueden atenderse tarde y el equipo tiene dificultades para identificar qué cambio pudo provocar un error.

Práctica: aplicar gestión de incidentes y trazabilidad de versiones, clasificando los reportes y registrando la versión afectada. La gestión de incidentes contempla detectar, analizar, priorizar y responder de manera organizada.

Evidencia: el caso indica que soporte recibe reportes, pero no los clasifica ni los relaciona con una versión.

3. Tres acciones prioritarias

1. Realizar pruebas de carga y concurrencia.

Es la primera acción porque el problema aparece con mayor demanda y permitiría detectar solicitudes duplicadas y errores de cupo. El costo sería tiempo, herramientas e infraestructura para simular usuarios. La mejora se comprobaría comparando tiempos de respuesta, errores y duplicaciones antes y después.

2. Centralizar los cambios y establecer una versión estable con posibilidad de rollback.

Debe hacerse después de detectar los problemas críticos para controlar los cambios y poder regresar a una versión funcional si ocurre un error. Nist señala que una configuración base ayuda a controlar versiones y cambios. La limitación sería el tiempo para organizar y documentar las versiones. Se comprobaría mediante una actualización y un regreso controlado a la versión anterior.

3. Organizar soporte y unificar las cifras de inscripción.

Los reportes deben tener prioridad y versión, y las áreas deben usar una misma fuente para los cupos. El costo sería tiempo de capacitación y coordinación. La mejora se comprobaría revisando que los reportes tengan prioridad, versión y datos consistentes.

4. Reflexión personal.

Este caso me hizo entender que desarrollar software no es nada más comprobar que funcione, también pensar en lo que puede pasar cuando muchos usuarios lo usan o cuando hay cambios. Considero importante realizar pruebas de carga, llevar un buen control de las versiones y documentar cambios. Así sería más fácil detectar errores, corregirlos y evitar que ocurran otra vez.

Uso de IA

Utilicé ChatGPT como apoyo para organizar las ideas e identificar los problemas del caso. Verifiqué la información con el caso y las fuentes consultadas, además de usarlo para corregir ortografía y redacción para hacerlo más entendible, y redacté el análisis final.

Referencias

Microsoft.(s. f.)¿Qué es AzureLoadTest?

<https://learn.microsoft.com/es-es/azure/app-testing/load-testing/overview-what-is-azure-load-testing>

National Institute(s. f.)ConfigurationBaseline.

https://csrc.nist.gov/glossary/term/Configuration_Baseline

National Institute(s. f.)IncidentResponse.

<https://csrc.nist.gov/projects/incident-response>

Microsoft(2024).Pruebas de carga y de esfuerzo de ASP.Core.

<https://learn.microsoft.com/es-es/aspnet/core/test/load-tests>